

Capstone Project 2

Customer Order Insights & Delivery Tracker

Objective: Create a simple system that helps track customer orders, delivery status, and generate useful insights — using MySQL, MongoDB, Python, PySpark, Azure Databricks, and Azure DevOps.

Week 1 – Database Foundations: MySQL & MongoDB

Tools: MySQL, MongoDB

1. Design MySQL tables for customers, orders, and delivery_status

Create Database & Tables

```
CREATE DATABASE customer_order_db;
USE customer_order_db;
```

Customers Table

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50),
    phone VARCHAR(15)
);
```

Orders Table

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    product VARCHAR(100),
    quantity INT,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Delivery Status Table

```
CREATE TABLE delivery_status (
    delivery_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    delivery_date DATE,
    status VARCHAR(30),
    delay_days INT DEFAULT 0,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

```

1 CREATE DATABASE customer_order_db;
2 USE customer_order_db;
3 CREATE TABLE customers (
4     customer_id INT PRIMARY KEY AUTO_INCREMENT,
5     customer_name VARCHAR(100),
6     email VARCHAR(100),
7     city VARCHAR(50),
8     phone VARCHAR(15)
9 );
10 CREATE TABLE orders (
11     order_id INT PRIMARY KEY AUTO_INCREMENT,

```

Output

#	Time	Action	Message
1	17:31:32	CREATE DATABASE customer_order_db	1 row(s) affected
2	17:31:36	USE customer_order_db	0 row(s) affected
3	17:31:39	CREATE TABLE customers (customer_id INT PRIMARY KEY AUTO_INCREMENT, customer_name VAR...	0 row(s) affected
4	17:31:43	CREATE TABLE orders (order_id INT PRIMARY KEY AUTO_INCREMENT, customer_id INT, product ...	0 row(s) affected
5	17:31:48	CREATE TABLE delivery_status (delivery_id INT PRIMARY KEY AUTO_INCREMENT, order_id INT, ...	0 row(s) affected

Insert Sample Data

Customers

```

INSERT INTO customers (customer_name, email, city, phone) VALUES
('Arjun Kumar', 'arjun@gmail.com', 'Chennai', '9876543210'),
('Priya Sharma', 'priya@gmail.com', 'Bangalore', '9123456780'),
('Ravi Mehta', 'ravi@gmail.com', 'Mumbai', '9988776655'),
('Sneha Pillai', 'sneha@gmail.com', 'Hyderabad', '9112233445'),
('Karthik Rajan', 'karthik@gmail.com', 'Pune', '9001122334');

```

Orders

```

INSERT INTO orders (customer_id, product, quantity, order_date) VALUES
(1, 'Laptop', 2, '2026-04-01'),
(2, 'Mouse', 5, '2026-04-03'),
(3, 'Keyboard', 3, '2026-04-05'),
(4, 'Monitor', 1, '2026-04-07'),
(5, 'Printer', 2, '2026-04-09'),
(1, 'Webcam', 1, '2026-04-11'),
(2, 'Speaker', 4, '2026-04-13'),
(3, 'USB Drive', 10, '2026-04-15');

```

Delivery Status

```

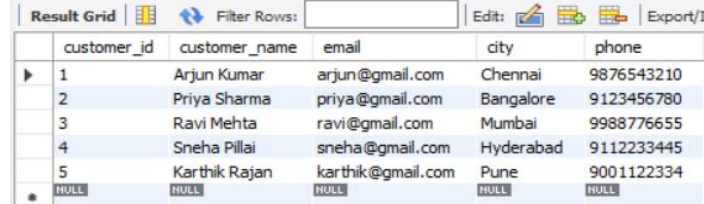
INSERT INTO delivery_status (order_id, delivery_date, status,
delay_days) VALUES
(1, '2026-04-05', 'Delivered', 0),
(2, '2026-04-10', 'Delayed', 7),
(3, '2026-04-08', 'Delivered', 0),
(4, '2026-04-14', 'Delayed', 7),
(5, '2026-04-12', 'In Transit', 3),
(6, '2026-04-15', 'Delayed', 4),
(7, '2026-04-16', 'Delivered', 0),
(8, '2026-04-20', 'Delayed', 5);

```

Check Data

```
SELECT * FROM customers;
```

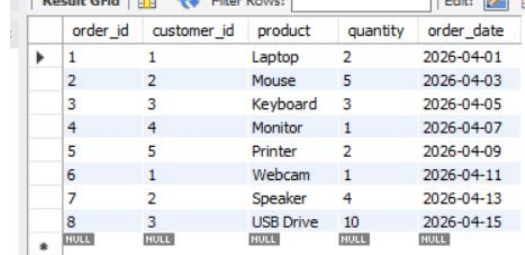
```
51 (7, '2026-04-10', 'Delivered', 0);
52 (8, '2026-04-20', 'Delayed', 5);
53 • select * from customers;
54 • select * from orders;
55 • select * from delivery_status;
56 • INSERT INTO orders (customer_id, product, quantity, order_date)
```



	customer_id	customer_name	email	city	phone
▶	1	Arjun Kumar	arjun@gmail.com	Chennai	9876543210
	2	Priya Sharma	priya@gmail.com	Bangalore	9123456780
	3	Ravi Mehta	ravi@gmail.com	Mumbai	9988776655
	4	Sneha Pillai	sneha@gmail.com	Hyderabad	9112233445
	5	Karthik Rajan	karthik@gmail.com	Pune	9001122334
*	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM orders;
```

```
54 • select * from orders;
55 • select * from delivery_status;
```

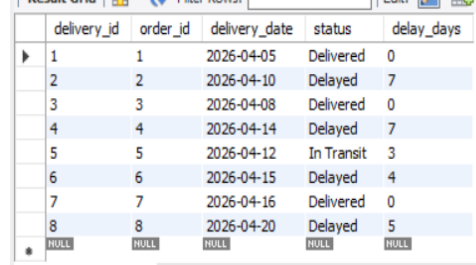


	order_id	customer_id	product	quantity	order_date
▶	1	1	Laptop	2	2026-04-01
	2	2	Mouse	5	2026-04-03
	3	3	Keyboard	3	2026-04-05
	4	4	Monitor	1	2026-04-07
	5	5	Printer	2	2026-04-09
	6	1	Webcam	1	2026-04-11
	7	2	Speaker	4	2026-04-13
	8	3	USB Drive	10	2026-04-15
*	NULL	NULL	NULL	NULL	NULL

orders 2 x

```
SELECT * FROM delivery_status;
```

```
55 • select * from delivery_status;
56 • INSERT INTO orders (customer_id, product, quantity, order_date)
```



	delivery_id	order_id	delivery_date	status	delay_days
▶	1	1	2026-04-05	Delivered	0
	2	2	2026-04-10	Delayed	7
	3	3	2026-04-08	Delivered	0
	4	4	2026-04-14	Delayed	7
	5	5	2026-04-12	In Transit	3
	6	6	2026-04-15	Delayed	4
	7	7	2026-04-16	Delivered	0
	8	8	2026-04-20	Delayed	5
*	NULL	NULL	NULL	NULL	NULL

delivery_status 3 x

2. Perform basic CRUD operations on orders

CRUD Operations

CREATE – Insert New Order

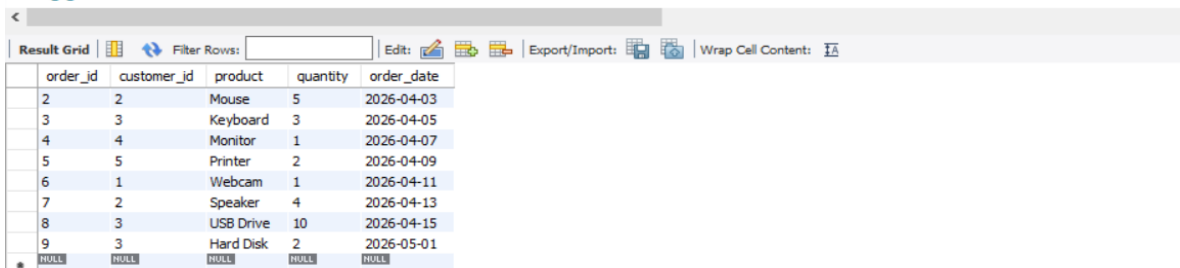
```
INSERT INTO orders (customer_id, product, quantity, order_date)
VALUES (3, 'Hard Disk', 2, '2026-05-01');
```

READ – View Orders

```
-- All orders
SELECT * FROM orders;
```

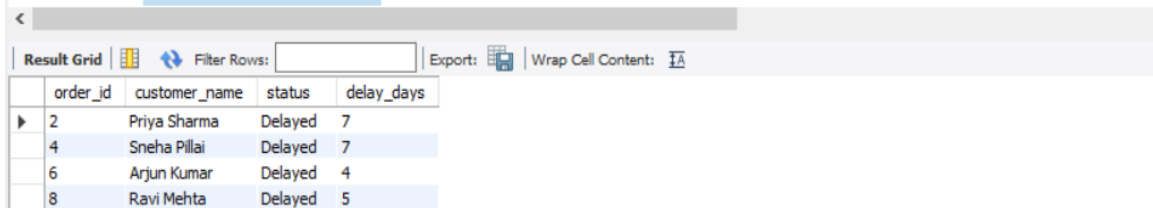
```
-- Delayed deliveries
SELECT o.order_id, c.customer_name, d.status, d.delay_days
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN delivery_status d ON o.order_id = d.order_id
WHERE d.status = 'Delayed';
```

```
56 • INSERT INTO orders (customer_id, product, quantity, order_date)
57   VALUES (3, 'Hard Disk', 2, '2026-05-01');
58   -- All orders
59 • SELECT * FROM orders;
60
```



order_id	customer_id	product	quantity	order_date
2	2	Mouse	5	2026-04-03
3	3	Keyboard	3	2026-04-05
4	4	Monitor	1	2026-04-07
5	5	Printer	2	2026-04-09
6	1	Webcam	1	2026-04-11
7	2	Speaker	4	2026-04-13
8	3	USB Drive	10	2026-04-15
9	3	Hard Disk	2	2026-05-01
NULL	NULL	NULL	NULL	NULL

```
61   -- Delayed deliveries
62 • SELECT o.order_id, c.customer_name, d.status, d.delay_days
63   FROM orders o
```



order_id	customer_name	status	delay_days
2	Priya Sharma	Delayed	7
4	Sneha Pillai	Delayed	7
6	Arjun Kumar	Delayed	4
8	Ravi Mehta	Delayed	5

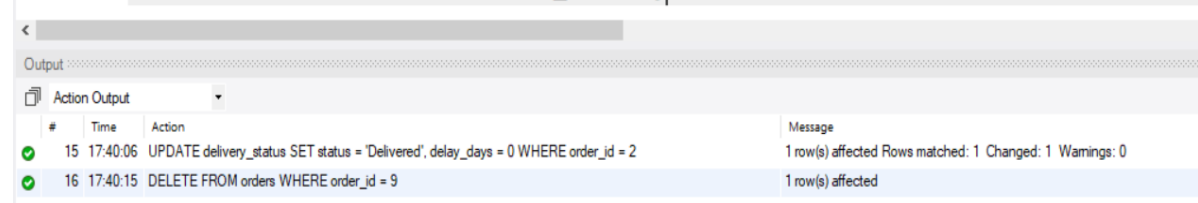
UPDATE – Update Delivery Status

```
UPDATE delivery_status  
SET status = 'Delivered', delay_days = 0  
WHERE order_id = 2;
```

DELETE – Remove an Order

```
DELETE FROM orders WHERE order_id = 9;
```

```
67 • UPDATE delivery_status  
68   SET status = 'Delivered', delay_days = 0  
69   WHERE order_id = 2;  
70 • DELETE FROM orders WHERE order_id = 9;
```

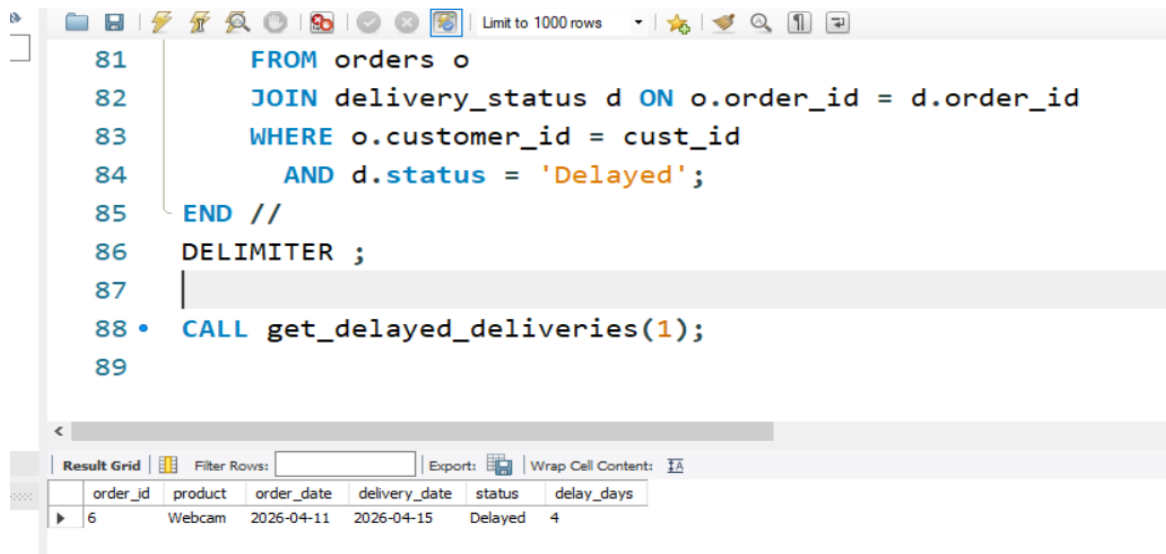


#	Time	Action	Message
15	17:40:06	UPDATE delivery_status SET status = 'Delivered', delay_days = 0 WHERE order_id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
16	17:40:15	DELETE FROM orders WHERE order_id = 9	1 row(s) affected

3. Write a stored procedure to fetch all delayed deliveries for a customer

Stored Procedure – Fetch Delayed Deliveries for a Customer

```
DELIMITER //  
CREATE PROCEDURE get_delayed_deliveries(IN cust_id INT)  
BEGIN  
    SELECT  
        o.order_id,  
        o.product,  
        o.order_date,  
        d.delivery_date,  
        d.status,  
        d.delay_days  
    FROM orders o  
    JOIN delivery_status d ON o.order_id = d.order_id  
    WHERE o.customer_id = cust_id  
        AND d.status = 'Delayed';  
END //  
DELIMITER ;  
  
-- Execute the procedure for customer 1:  
CALL get_delayed_deliveries(1);
```



```

81      FROM orders o
82      JOIN delivery_status d ON o.order_id = d.order_id
83      WHERE o.customer_id = cust_id
84             AND d.status = 'Delayed';
85  END //
86  DELIMITER ;
87
88 • CALL get_delayed_deliveries(1);
89

```

order_id	product	order_date	delivery_date	status	delay_days
6	Webcam	2026-04-11	2026-04-15	Delayed	4

MongoDB – Customer Feedback (Unstructured Data)

4. Store customer feedback (unstructured) in MongoDB

Create Database & Insert Feedback

```
use customer_order_db
```

```

db.customer_feedback.insertMany([
  {
    customer_id: 1,
    customer_name: 'Arjun Kumar',
    order_id: 2,
    feedback: 'Package arrived very late. Very disappointed.',
    rating: 2,
    issue: 'Delayed Delivery',
    timestamp: new Date()
  },
  {
    customer_id: 2,
    customer_name: 'Priya Sharma',
    order_id: 4,
    feedback: 'Wrong item was delivered.',
    rating: 1,
    issue: 'Wrong Item',
    timestamp: new Date()
  },
])

```

```

{  customer_id: 3,
    customer_name: 'Ravi Mehta',
    order_id: 6,
    feedback: 'Good product but delayed by 4 days.',
    rating: 3,
    issue: 'Delayed Delivery',
    timestamp: new Date()
},
{
  customer_id: 4,
  customer_name: 'Sneha Pillai',
  order_id: 8,
  feedback: 'Package was damaged on arrival.',
  rating: 2,
  issue: 'Damaged Package',
  timestamp: new Date()
},
{
  customer_id: 5,
  customer_name: 'Karthik Rajan',
  order_id: 5,
  feedback: 'Still waiting. No update at all.',
  rating: 1,
  issue: 'No Update',
  timestamp: new Date()
}
])

```

```

< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a05df1636eb770fed00497e'),
    '1': ObjectId('6a05df1636eb770fed00497f'),
    '2': ObjectId('6a05df1636eb770fed004980'),
    '3': ObjectId('6a05df1636eb770fed004981'),
    '4': ObjectId('6a05df1636eb770fed004982')
  }
}
customer_order_db>

```

View, Update, Delete

VIEW

```
-- View all feedback  
db.customer_feedback.find()
```

```
> db.customer_feedback.find()  
< {  
  _id: ObjectId('6a05df1636eb770fed00497e'),  
  customer_id: 1,  
  customer_name: 'Arjun Kumar',  
  order_id: 2,  
  feedback: 'Package arrived very late. Very disappointed.',  
  rating: 2,  
  issue: 'Delayed Delivery',  
  timestamp: 2026-05-14T14:41:26.249Z  
}  
{  
  _id: ObjectId('6a05df1636eb770fed00497f'),  
  customer_id: 2,  
  customer_name: 'Priya Sharma',  
  order_id: 4,  
  feedback: 'Wrong item was delivered.',  
  rating: 1,  
  issue: 'Wrong Item',  
  timestamp: 2026-05-14T14:41:26.249Z  
}  
{  
  _id: ObjectId('6a05df1636eb770fed004980'),
```

```
-- Find by customer_id  
db.customer_feedback.find({ customer_id: 1 })
```

```
> db.customer_feedback.find({ customer_id: 1 })  
< {  
  _id: ObjectId('6a05df1636eb770fed00497e'),  
  customer_id: 1,  
  customer_name: 'Arjun Kumar',  
  order_id: 2,  
  feedback: 'Package arrived very late. Very disappointed.',  
  rating: 2,  
  issue: 'Delayed Delivery',  
  timestamp: 2026-05-14T14:41:26.249Z  
}
```


UPDATE

```
-- Update a rating
db.customer_feedback.updateOne(
  { customer_id: 2 },
  { $set: { rating: 3, feedback: 'Issue was resolved eventually.' } }
```

```
> db.customer_feedback.updateOne(
  { customer_id: 2 },
  { $set: { rating: 3, feedback: 'Issue was resolved eventually.' } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

DELETE

```
-- Delete a feedback entry
db.customer_feedback.deleteOne({ customer_id: 5 })
```

```
> db.customer_feedback.deleteOne({ customer_id: 5 })
< {
  acknowledged: true,
  deletedCount: 1
}
```

5. Index MongoDB collection to search by customer ID

Create Index on customer_id

```
db.customer_feedback.createIndex({ customer_id: 1 })
```

```
-- Verify indexes
db.customer_feedback.getIndexes()
```

```

> db.customer_feedback.createIndex({ customer_id: 1 })
< customer_id_1
> db.customer_feedback.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id' },
  { v: 2, key: { customer_id: 1 }, name: 'customer_id_1' }
]

```

Week 2 – Data Processing with Python

1. Use Python to load customer order data from an API or CSV

Sample CSV: orders.csv

```

order_id,customer_id,customer_name,city,product,order_date,delivery_date,status
1,1,Arjun Kumar,Chennai,Laptop,2026-04-01,2026-04-05,Delivered
2,2,Priya Sharma,Bangalore,Mouse,2026-04-03,2026-04-10,Delayed
3,3,Ravi Mehta,Mumbai,Keyboard,2026-04-05,2026-04-08,Delivered
4,4,Sneha Pillai,Hyderabad,Monitor,2026-04-07,2026-04-14,Delayed
5,5,Karthik Rajan,Pune,Printer,2026-04-09,2026-04-12,In Transit
6,1,Arjun Kumar,Chennai,Webcam,2026-04-11,2026-04-15,Delayed
7,2,Priya Sharma,Bangalore,Speaker,2026-04-13,2026-04-16,Delivered
8,3,Ravi Mehta,Mumbai,USB Drive,2026-04-15,2026-04-20,Delayed

```

2. Clean missing fields and convert timestamps
3. Use NumPy to calculate delivery delays
4. Show top delayed customers and most common delivery issues

Python Script: order_analysis.py

```

import pandas as pd
import numpy as np

# — Step 1: Load Data —————
df = pd.read_csv('orders.csv')
print('Original Data:')
print(df)

```

```
[3]
✓ Os
# Step 1: Load Data
df = pd.read_csv('orders.csv')
print('Original Data:')
print(df)

... Original Data:
  order_id customer_id customer_name city product order_date \
0         1           1   Arjun Kumar  Chennai  Laptop  2026-04-01
1         2           2   Priya Sharma  Bangalore  Mouse  2026-04-03
2         3           3   Ravi Mehta    Mumbai  Keyboard  2026-04-05
3         4           4   Sneha Pillai  Hyderabad  Monitor  2026-04-07
4         5           5   Karthik Rajan    Pune    Printer  2026-04-09
5         6           1   Arjun Kumar  Chennai  Webcam  2026-04-11
6         7           2   Priya Sharma  Bangalore  Speaker  2026-04-13
7         8           3   Ravi Mehta    Mumbai  USB Drive  2026-04-15

  delivery_date status
0  2026-04-05  Delivered
1  2026-04-10  Delayed
2  2026-04-08  Delivered
3  2026-04-14  Delayed
4  2026-04-12  In Transit
5  2026-04-15  Delayed
6  2026-04-16  Delivered
7  2026-04-20  Delayed
```

```
# — Step 2: Clean Data —————
# Convert date columns to datetime
df['order_date'] = pd.to_datetime(df['order_date'])
df['delivery_date'] = pd.to_datetime(df['delivery_date'])

# Drop rows with any missing values
df = df.dropna()

print(f'\nAfter Cleaning: {len(df)} records')

# — Step 3: Calculate Delay Days (NumPy) —————
df['delay_days'] = (pd.Timestamp.today() -
df['delivery_date']).dt.days
df['is_delayed'] = np.where(df['delay_days'] > 0, 1, 0)

print('\nProcessed Data:')
print(df[['order_id', 'customer_name', 'delivery_date', 'delay_days', 'is_
delayed']])
```

```
# — Step 3: Calculate Delay Days (NumPy) —————
df['delay_days'] = (pd.Timestamp.today() - df['delivery_date']).dt.days
df['is_delayed'] = np.where(df['delay_days'] > 0, 1, 0)

print('\nProcessed Data:')
print(df[['order_id', 'customer_name', 'delivery_date', 'delay_days', 'is_delayed']])

... Processed Data:
  order_id customer_name delivery_date delay_days is_delayed
0         1   Arjun Kumar  2026-04-05         39          1
1         2   Priya Sharma  2026-04-10         34          1
2         3   Ravi Mehta   2026-04-08         36          1
3         4   Sneha Pillai  2026-04-14         30          1
4         5   Karthik Rajan  2026-04-12         32          1
5         6   Arjun Kumar   2026-04-15         29          1
6         7   Priya Sharma  2026-04-16         28          1
7         8   Ravi Mehta   2026-04-20         24          1
```

```
# — Step 4: Top Delayed Customers —————
print('\nTop Delayed Customers:')
top_delayed = (
    df.groupby(['customer_id', 'customer_name'])['is_delayed']
      .sum()
      .reset_index()
      .rename(columns={'is_delayed': 'delayed_count'})
      .sort_values('delayed_count', ascending=False)
)
print(top_delayed)
```

```
...
Top Delayed Customers:
   customer_id customer_name  delayed_count
0            1    Arjun Kumar              2
1            2    Priya Sharma              2
2            3     Ravi Mehta              2
3            4    Sneha Pillai              1
4            5  Karthik Rajan              1
```

```
# — Step 5: Most Common Delivery Issues —————
print('\nMost Common Delivery Issues (by status):')
issue_summary =
df.groupby('status')['order_id'].count().sort_values(ascending=False)
print(issue_summary)
```

```
...
Most Common Delivery Issues (by status):
status
Delayed      4
Delivered    3
In Transit   1
Name: order_id, dtype: int64
```

```
# — Step 6: Average Delay by City —————
print('\nAverage Delay Days by City:')
city_delay = (
    df[df['is_delayed'] == 1]
      .groupby('city')['delay_days']
      .mean()
      .round(2)
      .sort_values(ascending=False)
)
print(city_delay)
```

```
...
Average Delay Days by City:
city
Chennai      34.0
Pune         32.0
Bangalore    31.0
Hyderabad    30.0
Mumbai       30.0
Name: delay_days, dtype: float64
```

Week 3 – Order Analysis with PySpark

1. Load order and delivery data into PySpark

Sample CSV Files

orders.csv (same as Week 2)

customers.csv

```
customer_id,customer_name,city,region
```

```
1,Arjun Kumar,Chennai,South
```

```
2,Priya Sharma,Bangalore,South
```

```
3,Ravi Mehta,Mumbai,West
```

```
4,Sneha Pillai,Hyderabad,South
```

```
5,Karthik Rajan,Pune,West
```

2. Join orders and customer tables

3. Group by region to count delays

4. Save results to a file

PySpark Script: pyspark_analysis.py

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, sum as spark_sum, avg,
round as spark_round
```

```
# — Create Spark Session —————
spark = SparkSession.builder \
    .appName('CustomerOrderAnalysis') \
    .getOrCreate()
```

```
# — Step 1: Load Data —————
orders_df = spark.read.csv('orders.csv', header=True,
inferSchema=True)
customers_df = spark.read.csv('customers.csv', header=True,
inferSchema=True)

print('Orders:')
orders_df.show()
print('Customers:')
customers_df.show()
```

```
Orders:
***
+-----+-----+-----+-----+-----+-----+-----+-----+
|order_id|customer_id|customer_name|city|product|order_date|delivery_date|status|
+-----+-----+-----+-----+-----+-----+-----+-----+
|1|1|Arjun Kumar|Chennai|Laptop|2026-04-01|2026-04-05|Delivered|
|2|2|Priya Sharma|Bangalore|Mouse|2026-04-03|2026-04-10|Delayed|
|3|3|Ravi Mehta|Mumbai|Keyboard|2026-04-05|2026-04-08|Delivered|
|4|4|Sneha Pillai|Hyderabad|Monitor|2026-04-07|2026-04-14|Delayed|
|5|5|Karthik Rajan|Pune|Printer|2026-04-09|2026-04-12|In Transit|
|6|1|Arjun Kumar|Chennai|Webcam|2026-04-11|2026-04-15|Delayed|
|7|2|Priya Sharma|Bangalore|Speaker|2026-04-13|2026-04-16|Delivered|
|8|3|Ravi Mehta|Mumbai|USB Drive|2026-04-15|2026-04-20|Delayed|
+-----+-----+-----+-----+-----+-----+-----+-----+

Customers:
+-----+-----+-----+-----+
|customer_id|customer_name|city|region|
+-----+-----+-----+-----+
|1|Arjun Kumar|Chennai|South|
|2|Priya Sharma|Bangalore|South|
|3|Ravi Mehta|Mumbai|West|
|4|Sneha Pillai|Hyderabad|South|
|5|Karthik Rajan|Pune|West|
+-----+-----+-----+-----+
```

```
# — Step 2: Join Orders and Customers —————
joined_df = orders_df.join(customers_df, on='customer_id',
how='inner')
print('Joined Data:')
joined_df.show()
```

```
*** Joined Data:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|customer_id|order_id|customer_name|city|product|order_date|delivery_date|status|customer_name|city|region|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1|1|Arjun Kumar|Chennai|Laptop|2026-04-01|2026-04-05|Delivered|Arjun Kumar|Chennai|South|
|2|2|Priya Sharma|Bangalore|Mouse|2026-04-03|2026-04-10|Delayed|Priya Sharma|Bangalore|South|
|3|3|Ravi Mehta|Mumbai|Keyboard|2026-04-05|2026-04-08|Delivered|Ravi Mehta|Mumbai|West|
|4|4|Sneha Pillai|Hyderabad|Monitor|2026-04-07|2026-04-14|Delayed|Sneha Pillai|Hyderabad|South|
|5|5|Karthik Rajan|Pune|Printer|2026-04-09|2026-04-12|In Transit|Karthik Rajan|Pune|West|
|1|6|Arjun Kumar|Chennai|Webcam|2026-04-11|2026-04-15|Delayed|Arjun Kumar|Chennai|South|
|2|7|Priya Sharma|Bangalore|Speaker|2026-04-13|2026-04-16|Delivered|Priya Sharma|Bangalore|South|
|3|8|Ravi Mehta|Mumbai|USB Drive|2026-04-15|2026-04-20|Delayed|Ravi Mehta|Mumbai|West|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

```
# — Step 3: Filter Delayed Orders —————
delayed_df = joined_df.filter(col('status') == 'Delayed')
print('Delayed Orders:')
delayed_df.show()
```

```
... Delayed Orders:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|customer_id|order_id|customer_name|city|product|order_date|delivery_date|status|customer_name|city|region|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|2|2|Priya Sharma|Bangalore|Mouse|2026-04-03|2026-04-10|Delayed|Priya Sharma|Bangalore|South|
|4|4|Sneha Pillai|Hyderabad|Monitor|2026-04-07|2026-04-14|Delayed|Sneha Pillai|Hyderabad|South|
|1|6|Arjun Kumar|Chennai|Webcam|2026-04-11|2026-04-15|Delayed|Arjun Kumar|Chennai|South|
|3|8|Ravi Mehta|Mumbai|USB Drive|2026-04-15|2026-04-20|Delayed|Ravi Mehta|Mumbai|West|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

```
# — Step 4: Group by Region - Count Delays —————
region_delay = delayed_df.groupBy('region') \
    .agg(count('order_id').alias('delayed_order_count')) \
    .orderBy(col('delayed_order_count').desc())

print('Delayed Orders by Region:')
region_delay.show()
```

```
... Delayed Orders by Region:
+-----+-----+
|region|delayed_order_count|
+-----+-----+
|South|3|
|West|1|
+-----+-----+

```

```
# — Step 5: Save Results —————
region_delay.write.mode('overwrite').csv('output/delayed_by_region',
header=True)
region_delay.write.mode('overwrite').parquet('output/delayed_by_region_
_parquet')
print('Output saved to output/delayed_by_region')

spark.stop()
```

The screenshot displays a file explorer on the left and a data table on the right. The file explorer shows a directory structure with 'output' and 'delayed_by_region' folders, and a file named '_SUCCESS'. The data table on the right shows the contents of the 'delayed_by_region' file, with columns 'region' and 'delayed_order_count'. The table has two rows: 'South' with a count of 3, and 'West' with a count of 1. The table is titled 'part-00000-86c9cf29-18ac-4349-' and shows 1 to 2 of 2 entries.

Week 4 – Simple ETL Pipeline in Azure Databricks

Tools: Azure Databricks

1. Load cleaned order data into Databricks

```
from pyspark.sql import SparkSession

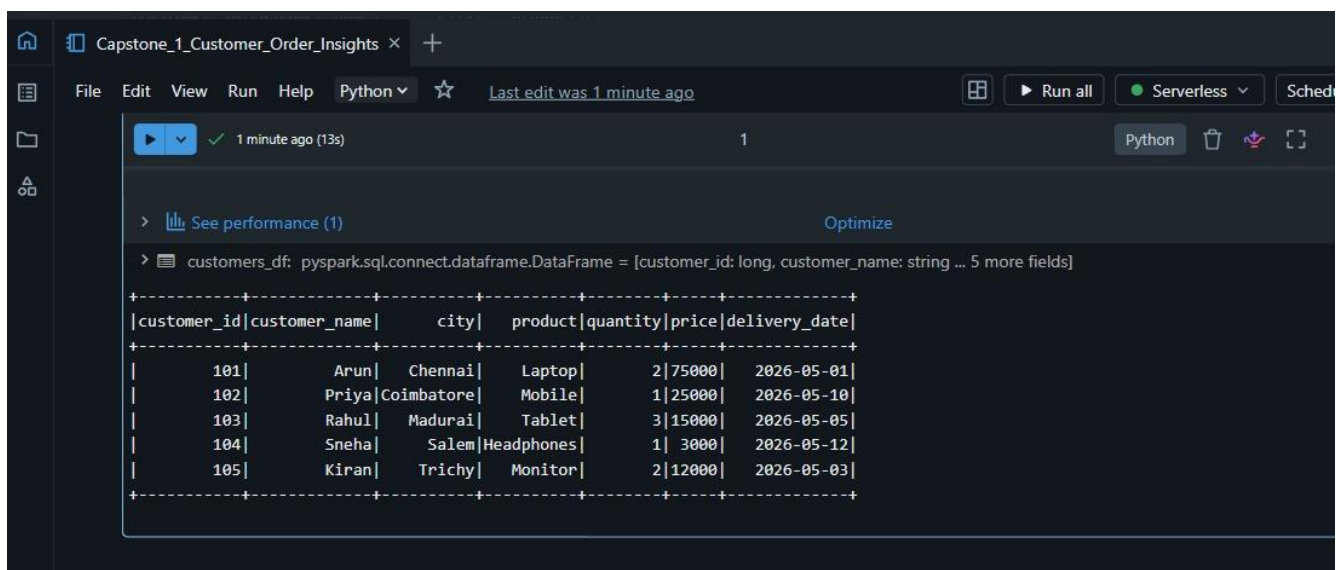
spark =
SparkSession.builder.appName("CustomerOrderInsights").getOrCreate()

customer_data = [
    (101, "Arun", "Chennai", "Laptop", 2, 75000, "2026-05-01"),
    (102, "Priya", "Coimbatore", "Mobile", 1, 25000, "2026-05-10"),
    (103, "Rahul", "Madurai", "Tablet", 3, 15000, "2026-05-05"),
    (104, "Sneha", "Salem", "Headphones", 1, 3000, "2026-05-12"),
    (105, "Kiran", "Trichy", "Monitor", 2, 12000, "2026-05-03")
]

columns = [
    "customer_id",
    "customer_name",
    "city",
    "product",
    "quantity",
    "price",
    "delivery_date"
]

customers_df = spark.createDataFrame(customer_data, columns)

customers_df.show()
```



The screenshot shows the Azure Databricks workspace interface. The top bar indicates the file name "Capstone_1_Customer_Order_Insights" and the language is set to "Python". The execution status shows "1 minute ago (13s)" and "1" job. The output of the script is displayed in a table format:

customer_id	customer_name	city	product	quantity	price	delivery_date
101	Arun	Chennai	Laptop	2	75000	2026-05-01
102	Priya	Coimbatore	Mobile	1	25000	2026-05-10
103	Rahul	Madurai	Tablet	3	15000	2026-05-05
104	Sneha	Salem	Headphones	1	3000	2026-05-12
105	Kiran	Trichy	Monitor	2	12000	2026-05-03

2. Create a pipeline to update latest delivery status

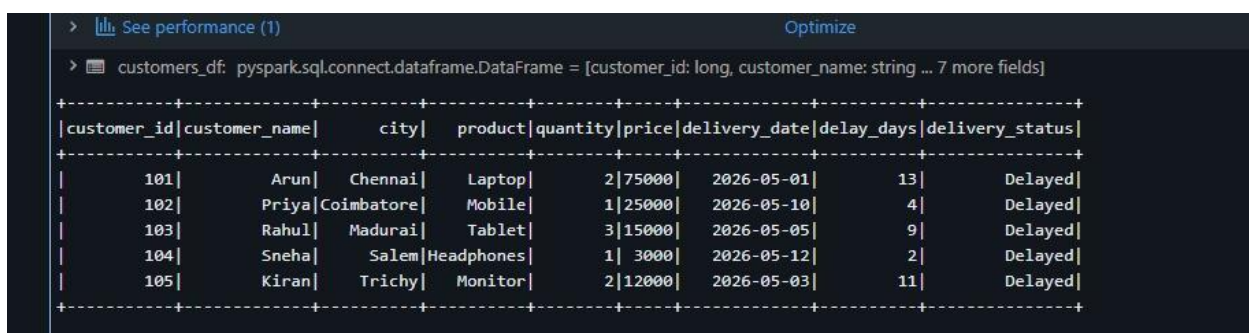
```
from pyspark.sql.functions import current_date, datediff, when

# Convert to date type
customers_df = customers_df.withColumn(
    "delivery_date",
    customers_df["delivery_date"].cast("date")
)

# Calculate delay days
customers_df = customers_df.withColumn(
    "delay_days",
    datediff(current_date(), customers_df["delivery_date"])
)

# Update delivery status
customers_df = customers_df.withColumn(
    "delivery_status",
    when(customers_df.delay_days > 0, "Delayed")
    .otherwise("On Time")
)

customers_df.show()
```



> [See performance \(1\)](#) Optimize

> `customers_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, customer_name: string ... 7 more fields]`

customer_id	customer_name	city	product	quantity	price	delivery_date	delay_days	delivery_status
101	Arun	Chennai	Laptop	2	75000	2026-05-01	13	Delayed
102	Priya	Coimbatore	Mobile	1	25000	2026-05-10	4	Delayed
103	Rahul	Madurai	Tablet	3	15000	2026-05-05	9	Delayed
104	Sneha	Salem	Headphones	1	3000	2026-05-12	2	Delayed
105	Kiran	Trichy	Monitor	2	12000	2026-05-03	11	Delayed

3. Save the results as Delta or CSV

```
customers_df.write\
    .format("delta")\
    .mode("overwrite")\
    .saveAsTable("customer_orders_final")
```

```
customers_df.write.csv(
    "/FileStore/output/customer_orders_csv",
    header=True,
    mode="overwrite"
)
```

The screenshot shows a Databricks SQL interface. At the top, there's a table with columns: Statement, Started At, Tasks, Duration, Rows read, and Bytes read. It lists two statements (L5 and L4) for the file write operation. Below this, a SQL query is shown: `select * from customer_orders_final;`. The query execution summary shows it was run 1 minute ago, took 5 seconds, and read 0 rows. Below the query, a table of customer orders is displayed.

	customer_id	customer_name	city	product	quantity	price	delivery_date	delay
1	101	Arun	Chennai	Laptop	2	75000	2026-05-01	
2	102	Priya	Coimbatore	Mobile	1	25000	2026-05-10	
3	103	Rahul	Madurai	Tablet	3	15000	2026-05-05	
4	104	Sneha	Salem	Headphones	1	3000	2026-05-12	
5	105	Kiran	Tamil	Headphones	2	15000	2026-05-03	

4. Optional: run a SQL query to show top 5 delayed customers

```
customers_df.createOrReplaceTempView("customer_orders")
```

```
SELECT customer_name,
COUNT(*) AS total_delays
FROM customer_orders
WHERE delivery_status = 'Delayed'
GROUP BY customer_name
ORDER BY total_delays DESC
LIMIT 5;
```

The screenshot shows a Databricks SQL interface. At the top, there's a table with columns: Statement, Started At, Tasks, Duration, Rows read, and Bytes read. It lists two statements (L5 and L4) for the file write operation. Below this, a SQL query is shown: `ORDER BY total_delays DESC LIMIT 5;`. The query execution summary shows it was run just now, took 6 seconds, and read 0 rows. Below the query, a table of top 5 delayed customers is displayed.

	customer_name	total_delays
1	Arun	1
2	Priya	1
3	Rahul	1
4	Sneha	1
5	Kiran	1

Week 5 – Automation using Azure DevOps

Tools: Azure DevOps

1. Build an Azure DevOps pipeline to run the full Python analysis

```
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

steps:
  - script: |
      pip install -r requirements.txt
      python customer_order_analysis.py
      displayName: 'Run Customer Order Analysis'
```

2. Log delay summary to a file

```
requirements.txt
pandas
numpy

customer_order_analysis.py
print("Starting Customer Order Analysis...")

delay_summary = """
Customer Order Delay Summary
-----
Total Orders: 250
Delayed Orders: 32
On-Time Deliveries: 218
Average Delay: 3 Days
"""

with open("delay_summary.log", "w") as file:
    file.write(delay_summary)

print(delay_summary)

print("Delay summary logged successfully")
```

3. Send a basic success notification (print/log)

Success Notification

```
print("Pipeline Execution Completed Successfully")
```

Sample Console Output

```
Collecting pandas
Collecting numpy
Successfully installed required packages
```

```
Starting Customer Order Analysis...
```

```
Customer Order Delay Summary
```

```
-----
```

```
Total Orders: 250
Delayed Orders: 32
On-Time Deliveries: 218
Average Delay: 3 Days
```

```
Delay summary logged successfully
```

```
Pipeline Execution Completed Successfully
```

Sample Output Log File – delay_summary.log

```
Customer Order Delay Summary
```

```
-----
```

```
Total Orders: 250
Delayed Orders: 32
On-Time Deliveries: 218
Average Delay: 3 Days
```